

AMF-Placer

第三天学习笔记

Cell Spreading: 从 Overflow 到可控的 Rough Legalization

今天只解决一个核心问题：

Quadratic Placement 可以把 HPWL 压得很低，但它并不保证局部资源需求不超过 FPGA 的实际供给。Cell Spreading 的任务，就是把过密区域中的实例逐步“摊开”，同时尽量不把线长重新破坏掉。



本日覆盖范围

- Section III-C: **Cell Spreading**
- Algorithm 1: AMF-Placer Cell Spreading
- Fig. 5: Macro 引起的 spreading deadlock
- 四个重点机制: resource supply fluctuation、utilization-guided window、two-phase macro spreading、forgetting rate

目录

1 先把 Cell Spreading 放回整个算法里	2
2 传统 partition-based spreading 的基本动作	3
3 Macro 为什么会造成 spreading deadlock?	4
4 机制一: Resource Supply Fluctuation	5
5 机制二: Utilization-Guided Spreading Window	6
6 机制三: Non-overlap Windows, 让 Cell Spreading 可以并行	7
7 机制四: Two-phase Macro Spreading	8
8 最值得理解的公式: Forgetting Rate	9
9 为什么 Forgetting Rate 能降低后期震荡?	10
10 Algorithm 1: 逐行看懂完整 Cell Spreading	11
11 把 Day 3 串成一条因果链	12

1 先把 Cell Spreading 放回整个算法里

Day 2 中, Quadratic Placement (QP) 主要负责线长优化。它产生的结果可视为论文中的 **lower-bound placement**: HPWL 较低, 但实例的位置仍是连续/近似的, 而且某些区域可能存在严重资源超额需求。

一句话定义

Cell Spreading = rough legalization。它不急着把每个 cell 精确地放到最终 BEL/site, 而是先解决“某一块区域里资源需求 > 资源供给”的问题。

论文把器件划分为 $N_Y \times N_X$ 个 bins:

$$B = \{B_{ij}\}.$$

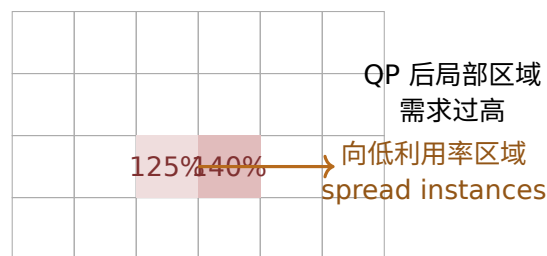
对某种资源 r , 可以用最基本的方式理解 bin 的利用率:

$$U_r(B_{ij}) = \frac{D_r(B_{ij})}{S_r(B_{ij})},$$

其中 D_r 是 demand, S_r 是 supply。当

$$U_r(B_{ij}) > 1$$

时, 就意味着这个 bin 对资源 r overflow。



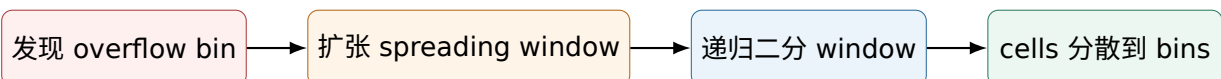
为什么不能只靠 QP?

QP 的目标函数主要在优化线长近似。即使所有连接都被拉得很紧, 也可能出现大量 LUT/FF/macro 聚集在某个局部区域, 而该区域的 CLB、DSP、BRAM 等物理资源数量是有限的。因此必须在 QP 之后加入 density/resource correction。

2 传统 partition-based spreading 的基本动作

论文所采用的基本框架来自常见的 bi-partitioning rough legalization。对于一个 overflowed bin, 大致执行:

1. 找到 overflowed bin;
2. 以它为中心向外扩张一个矩形 spreading window;
3. 确认这个 window 的总 resource supply 足以容纳其中的实例;
4. 递归二分该 window 及其中的 cells;
5. 把 cells 分散到 window 内不同 bins。



如果都是“单个小 cell”, 这个想法相对直接。但 AMF-Placer 的问题在于: **macro** 不是一个占用单一 **site** 的普通 **cell**。它可能跨越多个 sites, 内部带有形状约束, 而且可能与普通 cells 在 CLB 资源层面发生细粒度共享。

Day 3 的主矛盾

对于 macro, 移动不是“把一个点从 A 移到 B”, 而更像“搬动一块跨越多个 bins 的刚性结构”。你把它从一个 overflow 区域搬走, 可能马上在目标区域制造新的 overflow, 于是 spreading 可能反复震荡甚至形成 deadlock。

3 Macro 为什么会造成 spreading deadlock?

论文 Fig. 5 专门给出一个小例子：假设每个 bin 最多容纳 3 个 cells，但某个 macro 本身包含 5 个 cells。一次 spreading 看似把原来的 overflow 消掉了，但 macro 跨越多个 bins 后，又会把新的区域推到 overflow；下一轮 spreading 又把问题搬回来。

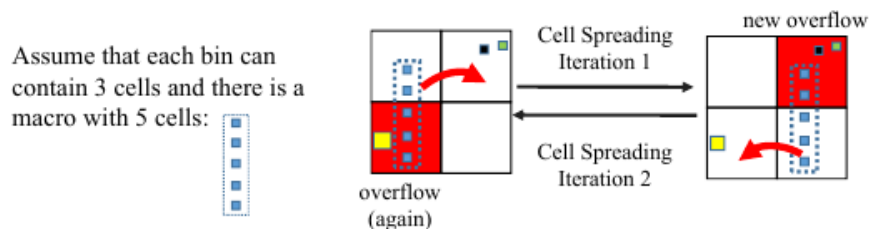


Fig. 5. An Example of Cell Spreading Deadlock of Macro

But sites with standard cells so AMF-Placer includes new

图 1: 论文 Fig. 5: Macro 引起的 cell spreading deadlock (直接转载自原论文第 5 页)。

把这个过程抽象成：

overflow at $A \rightarrow$ move macro $\rightarrow$ overflow at $B \rightarrow$ move again $\rightarrow \dots$

这不是简单的“算法没找到足够大的窗口”，而是 macro 的多-site footprint 使得局部容量约束相互耦合。

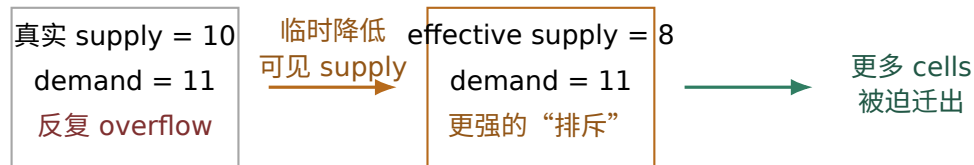
一个更直观的小例子

设每个 bin 有 3 个 slot, 两个相邻 bins 的剩余容量分别为 2 和 2。一个 5-cell macro 无论怎样跨在这两个 bins 上，总需求仍然是 5，而总剩余容量只有 4。若 spreading 只从局部视角把 macro 推向邻居，就很容易把 overflow 从一个位置转移到另一个位置，而不是消除它。

因此 AMF-Placer 没有只修改一个细节，而是针对 mixed-size spreading 同时加入多种稳定化机制。

4 机制一：Resource Supply Fluctuation

论文对反复 overflow 的 bins 记录 overflow 次数。若某个 bin 多次陷入 overflow, AMF-Placer 会对它周期性地降低并恢复 **resource supply**, 从而把一部分 instances “挤出去”。供给波动幅度与该 bin 被 overflow 的次数成正相关; 一旦该 bin 不再 overflow, fluctuation 立即停止。



这里要避免一个误解：论文不是在物理上真的减少 **FPGA** 资源。这是 spreading 过程内部使用的算法性 **supply** 调节，用于打破重复的局部状态。

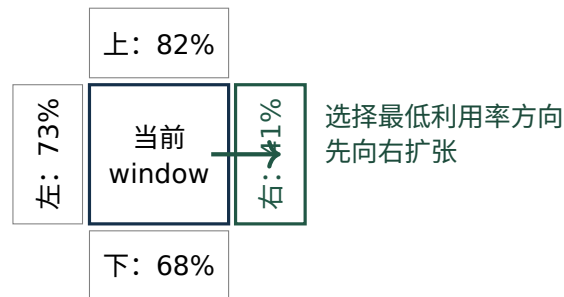
如何理解它的作用

如果一个区域总是“刚好还能塞一点”，算法可能反复把 cell 留在那里。临时把该区域看成更拥挤，相当于增加排斥力度，迫使更多实例离开，从而跳出 deadlock。

5 机制二：Utilization-Guided Spreading Window

传统方法中，spreading window 的搜索有两个典型问题：一种是枚举很多可能窗口，耗时；另一种是按照预先固定的方向/模式扩张，可能扩得过大，从而增加 HPWL。

AMF-Placer 的规则更直接：每次只检查当前窗口四个方向上的邻居 bins，然后优先向**利用率最低**的方向扩张，直到窗口拥有足够资源。



设四个方向的平均利用率为

$$U_{\text{left}}, U_{\text{right}}, U_{\text{up}}, U_{\text{down}},$$

则每一步可理解为选择

$$d^* = \arg \min_d U_d.$$

注意：这个 $\arg \min$ 是帮助理解论文策略的数学化表达，不是论文显式给出的公式。

为什么这有利于 HPWL?

目标不是找到“全芯片最空的地方”，而是尽量在当前 overflow 附近找到足够容量。优先朝低利用率邻居扩张，可以用较小的 window 解决局部 overflow，从而减少过度 spreading 和不必要的位移。

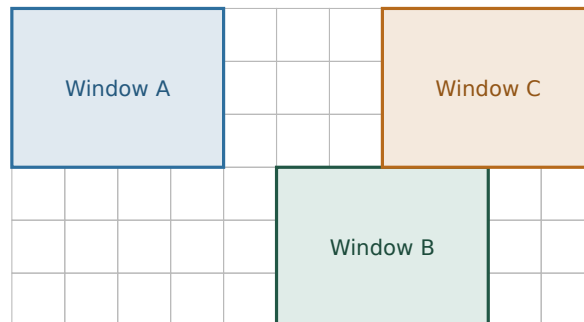
6 机制三：Non-overlap Windows, 让 Cell Spreading 可以并行

为了并行, 论文先按资源利用率对 overflowed bins 排序, 高利用率者优先确定 spreading region。某个 overflow bin 扩张 window 时, 会把窗口中的 bins 标记 (colored); 后续其他窗口不能覆盖这些已标记 bins。

最终得到一组互不重叠的 windows:

$$W_i \cap W_j = \emptyset, \quad i \neq j.$$

于是这些窗口内部的 standard-cell-level spreading 可以并行执行。



并行化为什么需要“不重叠”？

如果两个线程同时移动同一个 window 中的 cells, 或者同时消耗同一批 bin supply, 就会发生冲突。通过先构造空间上不重叠的 windows, 可以把多个局部 spreading 子问题拆成相互独立的并行任务。

这也是论文 Algorithm 1 中

```
parallel foreach Win in nonOverlapWins
```

可以成立的原因。

7 机制四：Two-phase Macro Spreading

对一个 spreading window，如果强制 window 必须完整覆盖其中所有 macro，遇到很多相邻的大 macro 时会把 window 扩得很大，导致 over-spreading 和 HPWL 增长。

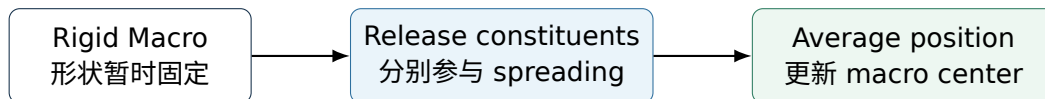
AMF-Placer 借鉴 mixed-size ASIC placer 的思想，把 macro spreading 拆成两个阶段：

1. 暂时释放 **shape constraint**：把 macro 内部的 standard cells 当作普通 cells 一样参与 spreading，用于解决资源 overflow；
2. 重建 **macro 位置**：用 macro 内部这些 standard cells 的平均位置更新 macro 的位置。

如果 macro M 包含 k 个 constituent cells，spread 后坐标为 (x'_j, y'_j) ，那么第二阶段可以直观理解成：

$$(x'_M, y'_M) = \left(\frac{1}{k} \sum_{j=1}^k x'_j, \frac{1}{k} \sum_{j=1}^k y'_j \right).$$

这正对应论文描述的“update to the average location of the standard cells in it”。



这里不是最终 legalization

释放 macro 的 shape constraint 只是 rough spreading 时的临时策略。它并不意味着最终允许 macro 内部 cells 分散在任意位置。真正的 shape/column constraint 会在后续 Progressive Macro Legalization 中逐步恢复并最终严格满足。

8 最值得理解的公式：Forgetting Rate

Cell spreading 有一个副作用：它为了消除 overflow，可能把原本已经很紧凑、线长很好的 placement 拉得太开。论文因此没有直接采用 spreading 算出来的新位置 (x'_i, y'_i) ，而是和上一次 upper-bound 位置做插值：

$$(x_i^{U'}, y_i^{U'}) = (\gamma x'_i + (1 - \gamma)x_i^U, \gamma y'_i + (1 - \gamma)y_i^U)$$

并设置

$$\gamma = 1 - 0.95 \cdot \frac{HPWL_{lower}}{HPWL_{upper}}$$

rithm 1), resource overflows will be gradually resolved.

Finally, the partitioning-based cell spreading algorithm is sensitive to the overflow. When a placement is compact, a single overflowed bin could cause cell spreading in a large region and seriously increase wirelength. Therefore, instead of directly updating instance location, we keep the location of an instance v_i in last upper-bound placement as (x_i^U, y_i^U) . During cell spreading, when the instance is expected to spread to the location (x'_i, y'_i) , the actual updated location will be set to:

$$(\gamma x'_i + (1 - \gamma)x_i^U, \gamma y'_i + (1 - \gamma)y_i^U)$$

where γ is forgetting rate and set to be $(1 - 0.95 \times HPWL_{lower}/HPWL_{upper})$ in AMF-Placer (line 11 in Algorithm 1). It can gradually depress the sensitivity of

图 2: 原论文第 6 页中关于 forgetting rate 的公式与解释 (截取)。

把这个公式慢慢拆开

因为通常有

$$HPWL_{lower} \leq HPWL_{upper},$$

所以比值

$$r = \frac{HPWL_{lower}}{HPWL_{upper}}$$

可以看成 upper/lower placement 已经靠得多近。

如果还没收敛，例如 $r = 0.60$ ：

$$\gamma = 1 - 0.95 \times 0.60 = 0.43.$$

新 spreading 位置的权重还有 43%。

如果已经很接近，例如 $r = 0.98$ ：

$$\gamma = 1 - 0.95 \times 0.98 = 0.069.$$

新 spreading 位置只占约 6.9%，大部分保留旧 upper-bound 位置。

9 为什么 Forgetting Rate 能降低后期震荡?

把一维情况写成

$$x^{U'} = x^U + \gamma(x' - x^U).$$

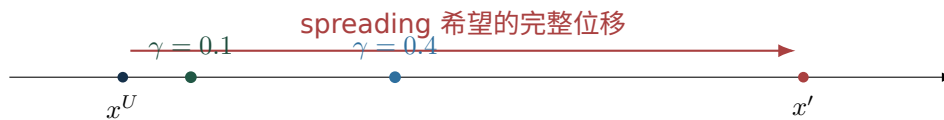
其中

$$\Delta x_{spread} = x' - x^U$$

是 spreading 本来想施加的位移, 因此实际位移是

$$\Delta x_{actual} = \gamma \Delta x_{spread}.$$

这就清楚了: γ 本质上是一个 **step-size / damping factor**。



当 global placement 尚未收敛时, 可以允许 cell spreading 更积极地修正 density; 当 lower-bound 与 upper-bound 的 HPWL 已经很接近时, 说明算法接近稳定, 此时若仍然大幅 spreading, 很容易破坏已有线长, 所以 γ 自动变小。

这一点与 Day 2 的 pseudo-net 思想很相似

Day 2 中 pseudo net 用来限制 QP 每次不要跳得太远; Day 3 中 forgetting rate 用来限制 spreading 后期不要修得太猛。两者都在做同一件事: **给迭代优化加入稳定性和渐进性**。

10 Algorithm 1: 逐行看懂完整 Cell Spreading

require high density of resource, span multiple sites and share

Algorithm 1: AMF-Placer Cell Spreading

Input: instance locations in lower-bound placement
 $\mathbf{P}^L = \{(x_i^L, y_i^L)\}$, instance locations in last
upper-bound placement $\mathbf{P}^U = \{(x_i^U, y_i^U)\}$, forgetting
rate γ

Output: instance locations in new upper-bound placement
 $\mathbf{P}^{U'} = \{(x_i^{U'}, y_i^{U'})\}$

- 1 OFBins = findAndSortOverflowBins($\mathbf{P}^L$);
- 2 OFCells = cells in OFBins;
- 3 resetOverflowTimes(OFBins);
- 4 **while** $size(OFCells) > \theta_{of}$ **do**
- 5 updateOverflowTimes(OFBins);
- 6 injectSupplyFluctuation(OFBins);
- 7 nonOverlapWins =
- 6 greedilyFindNonOverlapWindow(OFBins);
- 8 **parallel foreach** Win in $nonOverlapWins$ **do**
- 9 $\mathbf{P}'_{cell} = cellSpreading(Win)$; // update locally
- 10 $\mathbf{P}' = updateMacroLocation(\mathbf{P}'_{cell})$;
- 11 $\mathbf{P}^{U'} = updateLBPlacementWith\gamma(\mathbf{P}')$;
- 12 OFBins = findAndSortOverflowBins($\mathbf{P}^{U'}$);
- 13 OFCells = cells in OFBins;
- 14 **adjustResourceDemandSupply()**;

图 3: 论文 Algorithm 1: AMF-Placer Cell Spreading (直接截取自原论文第 5 页)。

行号	含义
1-3	从 lower-bound placement 找到并排序 overflow bins, 收集其中 cells, 并重置 overflow 次数。
4	只要 overflow cells 数量仍超过阈值 θ_{of} , 继续 spreading。
5-6	更新每个 bin 的 overflow 次数, 并对反复 overflow 的区域注入 supply fluctuation。
7	为 overflow bins 贪心构造互不重叠的 spreading windows。
8-9	对各个 non-overlap window 并行执行局部 cell spreading。
10	根据 constituent cells 的位置更新 macro location。
11	使用 forgetting rate γ 生成新的 upper-bound placement。
12-13	重新检测 overflow, 准备下一轮。
14	根据 packing/routing feasibility 调整 resource demand/supply。

11 把 Day 3 串成一条因果链



今天应当真正记住的 4 句话

1. Cell spreading 不是 final legalization, 而是处理局部 resource overflow 的 rough legalization。
2. Macro 会把 spreading 从“点移动”变成“多-site footprint 移动”, 因此容易制造新的 overflow 和 deadlock。
3. AMF-Placer 不只追求“能 spread 开”, 还追求 **window** 尽量小、可并行、**macro** 位移不过度。
4. Forgetting rate 让后期更新越来越保守, 是整个 flow 收敛稳定性的关键配件之一。

下一天预告: Progressive Macro Legalization

下一天将进入 Section III-D。重点会是: 为什么 macro legalization 是 1-to-many、rough legalization 为什么先把 macro constituent cells 拆开做 bipartite matching、potential legal location 如何变成 pseudo-net anchor, 以及什么时候从 rough legalization 切换到 exact legalization, 最后再推导 intra-column DP:

$$f(i, j) = \min \left(f(i-1, j - \text{row}(i)) + HPWL^+(i, j - \text{row}(i) + 1), f(i, j-1) \right).$$